VitaMate AI Food Analysis Service

Academic Technical Report on AI Service Implementation, Training Methodology, Runtime Pipeline, and Experimental Evaluation

Prepared for Graduation Committee Review

Source basis: current public GitHub repository + project training-history documentation

Repository state reviewed: 18 August 2026

GitHub Repository: amenahzitoun-ENG/AI-Food-Analysis

Abstract

This report presents a detailed technical and academic account of the implementation of the VitaMate AI Food Analysis Service. The service is designed as a local, finalize-first, human-in-the-loop food-analysis pipeline for top-down images containing a single food container. The implementation combines food-region segmentation, closed-set semantic dish recognition, catalog-conditioned ingredient suggestion, optional monocular-depth-based weight estimation, and deterministic nutrition calculation. The report reconstructs the development and training process from the project training history, verifies the present deployment state against the current GitHub repository, explains the runtime architecture under src/vitamate_ai_package, and reports the measured outcomes of the principal experimental phases. Particular attention is given to reproducibility, model promotion governance, separation between experimental performance and active deployment state, and limitations caused by small evaluation sets and domain shift. The current authoritative runtime uses an active YOLO-based segmentation model with a rescue model, SigLIP2 in zero-shot closed-set mode for semantic ranking, catalog-first ingredient suggestions, manual weight as the reliable default, optional Depth Anything V2 metric depth for diagnostic automatic weight estimation, and final nutrition only after user confirmation through the /finalize endpoint [1]-[3].

The historical training record documents several additional experiments, including two-stage segmentation fine-tuning on 59 human masks, deterministic weight calibration using measured VitaMate weights, SigLIP2 Mini100 zero-shot evaluation, and independent dish/ingredient fine-tuning. These experiments provide evidence for design decisions but must not be conflated with the current active model registry. The current GitHub registry explicitly states that newer final segmentation candidates were not retained and that fine-tuned semantic checkpoints are inactive unless their independent promotion gates pass [2], [3].

Keywords: food image analysis, YOLO segmentation, SigLIP2, Depth Anything V2, nutrition estimation, computer vision, human-in-the-loop AI, model governance, FastAPI, VitaMate.

Source Reconciliation Statement

Important academic distinction: The internal training-history document and the current GitHub repository represent different project states. Historical experiment tables are therefore reported as historical evidence. Deployment claims follow the current README, CURRENT_STATE.md, and MODEL_REGISTRY.md. For example, the historical Phase 1 snapshot reports seed17 as the strongest human-mask candidate, whereas the current model registry lists foodv1_finetune_quality.pt as the active primary and states that new final candidates were rejected/not retained [2], [3].

Table of Contents

1. Introduction and Service Scope

1.1 Motivation and problem definition

Automatic nutritional understanding from a single meal image is a multi-stage computer vision problem. A system must first determine which pixels correspond to food, then infer a semantically meaningful dish hypothesis, estimate or request the physical quantity of food, map the confirmed content to a nutrient database, and control uncertainty so that unreliable intermediate predictions are not presented as final nutrition. Nutrition5k demonstrates the broader research motivation by providing approximately five thousand real dishes with visual, component-weight, depth, and nutritional annotations, illustrating the importance of combining appearance and geometric information for food understanding [7].

VitaMate does not attempt unrestricted open-world food recognition. The current public repository defines the service as a local, finalize-first MVP for top-down, single-container meal images. Its official flow is: food-region segmentation, dish Top-3 suggestion, catalog-first ingredient suggestion, user confirmation of dish/ingredients/weight, and final nutrition calculation. Automatic weight is explicitly optional and is skipped by the browser demo by default [1], [2].

1.2 Functional objectives of the AI service

- Locate the food region reliably enough to support downstream semantic and geometric analysis.
- Return closed-set dish candidates rather than claim unrestricted food recognition.
- Condition ingredient suggestions on the curated project catalog and require user confirmation.
- Provide an optional geometric weight estimate when scale and depth are sufficiently reliable, while preserving manual weight as the primary fallback.
- Calculate final nutritional values only after the user confirms the semantic content and total meal weight.
- Keep training, evaluation, promotion, and runtime activation as separate concerns so that an experiment cannot silently replace an active model.

1.3 Supported capture protocol

Table 1. Current supported scope derived from the repository README and current-state document [1], [2].

Property	Supported/Required behavior
Camera orientation	Top-down or sufficiently close to the supported top-down protocol
Containers	One primary food container
Recognition space	Curated closed-set dish catalog
Semantic backbone	Local SigLIP2
Weight	Manual by default; optional metric-depth path
Nutrition	Final values only after explicit confirmation
Open-world claims	Not supported

2. Repository Architecture and Implementation Organization

The repository separates the operational package from one-off or reproducible training/evaluation scripts. This separation is academically important because it prevents the runtime API from becoming coupled to experiment orchestration. In practical terms, scripts/ contains data preparation, training, evaluation, calibration, promotion, and reproducibility utilities, while src/vitamate_ai_package contains code imported by the running service. Model manifests and active-state records are under models/, and generated experimental evidence is written under results/ and runs/. The repository intentionally excludes large datasets, local model weights, generated runs, and the virtual environment from Git [1].

Table 2. High-level repository organization.

Repository area	Primary responsibility	Examples
src/vitamate_ai_package	Runtime package and reusable AI components	api, pipeline, segmentation, classification, depth, weight, nutrition
scripts	Experiment and lifecycle orchestration	dataset preparation, training, evaluation, promotion
models	Local active/optional checkpoints and manifests	segmentation active model, semantic activation manifest
results/current	Canonical evaluation and promotion evidence	release evaluation, calibration reports
runs	Generated training runs/checkpoints	YOLO and semantic fine-tuning outputs
Data	Local datasets/manifests (excluded from Git)	human masks, processed semantic crops

Figure 1. Architectural separation of the deployed request path.

Image/API request → Runtime package (src) → Model assets + vocab/catalog → Analysis session → User confirmation → Final nutrition

2.1 Experimental lifecycle versus runtime lifecycle

The training lifecycle generates candidates and evaluation evidence; it does not directly replace active models. The current model registry states that training never changes the active model and that activation requires a successful promotion decision together with existing checkpoint files [3]. This governance principle is reflected in the use of active manifests, hashes, promotion reports, and separate experimental directories.

Simplified experiment-to-deployment lifecycle.

```
DATA / MANIFEST -> TRAIN CANDIDATE -> INDEPENDENT EVALUATION -> PROMOTION GATE -> ACTIVE MANIFEST
                                                |
                                                +-> reject / retain evidence only
```

3. Data Governance and Dataset Preparation

3.1 Phase 0: measured VitaMate data governance

The project training record documents a governance phase designed to establish data integrity before model adaptation. Three principal scripts were used: import_vitamate_labeled_60.py to import the measured project images and metadata, validate_vitamate_metadata.py to validate the schema and required fields, and prepare_vitamate_phase0_governance.py to audit image identity, measured weight availability, capture protocol, human masks, and split leakage [4].

Table 3. Phase 0 governance results from the project training record [4].

Governance measure	Result
Rows	60
Valid images	60
Unique image hashes	60
Measured weights	60 / 60
Protocol reviewed	60 / 60
Valid human masks	59 / 60
Human train split	40
Human validation split	10
Human test split	9
Overlap holdout	1 case isolated from training

The explicit use of hashes and split checks is important because image-level leakage would invalidate a small test set. The governance stage also distinguishes measured ground truth from estimated labels, which is essential for the weight-calibration stage.

3.2 Phase 1: human-mask segmentation dataset

Food masks were created through a local annotation interface (annotate_vitamate_human_masks.py). The preparation script prepare_vitamate_phase1_segmentation_dataset.py converted human binary masks into a YOLO-compatible single-class segmentation dataset and generated train/validation/test artifacts. The final task is deliberately constrained to one class, food, which is also enforced by the training script when it validates the dataset YAML [4].

Dataset transformation used for the human-mask segmentation experiment.

```
Human binary mask -> contours/polygons -> normalized YOLO segmentation labels -> train/val/test YAML
```

3.3 Semantic dataset preparation

The later semantic phases used prepare_vitamate_final_semantic_dataset.py to generate semantic crops/manifests while controlling split leakage, and build_vitamate_dish_ingredient_mapping.py to construct canonical dish-to-ingredient mappings. The historical Phase 3A record contained 440 semantic rows (355 train, 56 validation, 29 human test), initially covering six trainable dishes plus an unknown class and eight trainable ingredients [4]. A later mapping-aware phase organized 60 VitaMate images into ten canonical dishes and 48 unique ingredients with no missing nutrition mappings in that snapshot [4].

4. Segmentation Training and Evaluation

4.1 Initializer and training strategy

The final human-mask experiment did not train a segmentation network from scratch. The script train_vitamate_final_segmentation.py initializes from models/segmentation/official/foodv1_finetune_quality.pt and performs a controlled two-stage fine-tuning recipe over two random seeds (17 and 29). The current script also computes SHA-256 hashes of the dataset and initializer, records the dataset fingerprint, supports a one-epoch preflight for memory safety, and applies conservative augmentation [5].

Table 4. Historical Phase 1 execution settings [4], consistent with the current training script defaults [5].

Stage	Epochs	Frozen layers	Initial LR	Purpose
Warm stage	5	15	1e-4	Adapt task head/upper network conservatively before full unfreezing
Controlled unfreeze	15	0	2e-5	Fine-tune the complete model with a smaller learning rate

The script selects a low-VRAM hardware profile when GPU memory is less than or equal to approximately 3 GB: image size 512, batch size 1, and AMP disabled. Otherwise it defaults to image size 640, batch size 4, and AMP enabled. AdamW is used, deterministic training is requested, patience is five epochs, and aggressive augmentations such as mosaic, mixup, and copy-paste are disabled in this final adaptation recipe [5].

Simplified two-stage segmentation training logic (adapted from train_vitamate_final_segmentation.py).

```
for seed in [17, 29]:
    warm_best = train(initializer, freeze=15, epochs=5, lr=1e-4)
    final_best = train(warm_best, freeze=0, epochs=15, lr=2e-5)
    save_manifest(seed, warm_best, final_best, dataset_fingerprint)
```

4.2 Independent human-mask evaluation

The evaluation script evaluates each checkpoint against human binary masks using the same image/mask pairs, measures inference latency, saves per-image metrics and visual overlays, and ranks models by mean IoU [6]. The principal pixel-level metrics are defined below.

$$IoU = TP / (TP + FP + FN)$$

$$Dice = 2TP / (2TP + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

Here TP is the number of food pixels correctly predicted as food, FP is background incorrectly included as food, and FN is true food omitted by the model. Leakage ratio and fragmentation error provide additional diagnostics for background spill and mask fragmentation.

Table 5. Historical nine-image human-mask evaluation [4].

Model	Mean IoU	Median IoU	Mean Dice	Precision	Recall	Mean latency
foodv1_finetune_quality	85.27%	91.15%	90.52%	97.51%	87.45%	322.67 ms
seed17_final	90.74%	91.47%	95.01%	95.58%	94.96%	126.68 ms
seed29_final	90.01%	91.52%	94.58%	94.97%	94.83%	167.50 ms

Within this historical test snapshot, seed17 improved mean IoU by 5.47 percentage points, Dice by 4.49 points, and recall by 7.50 points relative to foodv1, while leakage increased from 2.41% to 4.68%. This illustrates a typical precision-recall trade-off: greater coverage of the true food region can increase minor spill beyond the ground-truth boundary [4].

4.3 Promotion-state reconciliation

Historical result versus current deployment: The project training-history snapshot records seed17 as the selected primary candidate after the nine-image experiment. The current GitHub model registry, however, lists foodv1_finetune_quality.pt as the active official primary, baseline_food_seg_subset.pt as rescue-only, and states that new final candidates were rejected/not retained [2], [3]. Consequently, seed17 is reported in this document as historical experimental evidence, not as the current deployed model.

4.4 Runtime segmentation behavior

At runtime, FoodSegmenter loads the active and rescue segmentation configurations, converts YOLO masks to a common boolean mask representation, removes small disconnected components, can use detected plate geometry to clip masks, and can combine sufficiently consistent primary/rescue masks. The runtime package also retains heuristic fallbacks for failure handling; however, automatic weight is explicitly disabled when the mask originates from the heuristic fallback [2].

5. Automatic Weight Estimation and Depth Anything V2

5.1 Why the weight stage is deterministic rather than a trained weight network

The VitaMate weight stage is not a separately trained neural network. It is a deterministic estimator that combines the food mask, a physical pixel-to-centimeter scale, metric depth, density rules, and a calibrated shape factor. The historical calibration script consumes baseline CSV rows that already contain measured weights and depth/volume diagnostics; it then evaluates whether changing the shape factor is justified [4], [10].

Figure 2. Deterministic automatic-weight calculation path.

Food mask → Scale resolution (cm/pixel) → Metric depth map → Food height above plate → Raw volume → Shape-factor correction → Density → Estimated weight

5.2 Role of Depth Anything V2

Depth Anything V2 is used only when automatic weight is requested. The current repository uses a metric-depth checkpoint and loads it through MetricDepthEstimator. The wrapper imports the local metric-depth implementation, loads the configured checkpoint into PyTorch, switches the model to evaluation mode, and produces a depth map in meters under torch.inference_mode() [2], [11]. The Depth Anything V2 paper describes the model family as a robust monocular depth-estimation approach and reports improved fine-grained depth prediction relative to its predecessor [9].

The model is not fine-tuned by VitaMate. It is an inference component. The current model registry lists the metric-depth checkpoint under Depth-Anything-V2/metric_depth/checkpoints and states that it is used only when auto_weight_mode=try [3].

5.3 Geometry and weight equation

The runtime estimator first resolves a scale in centimeters per pixel from plate geometry, a dishware profile, an explicit plate diameter, or a user-provided reference. It then estimates a representative plate depth from pixels around or outside the food mask. Food height is calculated as the positive difference between the plate depth and each food pixel's depth. Each pixel contributes a small physical volume equal to height times pixel area. The raw volume is then corrected by a shape factor and multiplied by density [12].

$$h_i(cm) = \max((d_{plate} - d_i) \times 100, 0)$$

$$A_{pixel}(cm^2) = (scale\_cm\_per\_pixel)^2$$

$$V_{raw}(cm^3) = \Sigma_{i \in food\ mask} h_i \times A_{pixel}$$

$$V_{corrected} = V_{raw} \times shape\_factor$$

$$Estimated\ weight\ (g) = V_{corrected} \times density\ (g/cm^3)$$

The estimator contains guards for low segmentation confidence, unsupported container geometry, implausible mask coverage, missing scale, inconsistent depth reference, very low inferred heights, and implausibly high heights. These guards are essential because monocular depth and scale estimation can produce numerically plausible but physically misleading weight values [12].

5.4 Shape-factor calibration and measured results

The calibration script fits two alternative factors from the training rows: a least-squares factor and a median ratio factor. For a reliable row, x equals raw volume multiplied by density, while y equals measured weight. The least-squares factor minimizes squared error through the closed-form one-parameter regression solution, and the median-ratio factor is the median of y/x across eligible rows [10]. A new factor is recommended only if it improves validation MAE by at least 25 g according to the script defaults.

Table 6. Historical VitaMate weight-calibration comparison [4].

Candidate factor	Validation MAE	Validation Median AE	Test MAE	Test Median AE

Candidate factor	Validation MAE	Validation Median AE	Test MAE	Test Median AE
Current 0.347797	117.38 g	78.53 g	164.80 g	122.04 g
Least-squares 0.370999	148.01 g	93.68 g	164.67 g	104.46 g
Median-ratio 0.451223	253.92 g	201.77 g	165.00 g	125.68 g

Because the current factor achieved the lowest validation MAE, the historical decision was keep_current_shape_factor. After the final depth guard on the nine-image test split, valid auto-weight was 8/9 (88.89%), MAE was 163.21 g, median absolute error was 125.22 g, and MAPE was 36.40%. The project therefore retained manual weight as the trusted product path [4]. The current GitHub state further notes that the latest 35-image runtime suite has no measured ground-truth weights, so automatic weight is not officially evaluated on that set [2].

6. Semantic Recognition: SigLIP2 Experiments and Fine-Tuning

6.1 Why SigLIP2 was selected as the semantic backbone

SigLIP2 is a multilingual vision-language encoder that produces compatible representations for images and text. The SigLIP2 research reports improved zero-shot classification, image-text retrieval, and transfer performance over SigLIP at multiple scales [8]. This capability makes it suitable for a closed-set system in which an image embedding can be compared with text embeddings representing dish labels or prompt templates.

6.2 Mini100 zero-shot experiment

The Mini100 study compared a frozen SigLIP2 zero-shot baseline with a linear dish head trained on frozen SigLIP2 embeddings. The dataset contained 100 images split into 66 train, 17 validation, and 17 test samples, with seven known classes plus an unknown class [4]. In the zero-shot script, text prompts and images are independently encoded and L2-normalized. Similarity is obtained from the matrix product of image and text features; the validation split is used to calibrate unknown-rejection thresholds, while the test set is evaluated once [13].

Simplified zero-shot semantic scoring and rejection logic.

```
text_vecs = normalize(encode_text(prompt_templates))
image_vec = normalize(encode_image(image))
logits = image_vec @ text_vecs.T / temperature
probabilities = softmax(logits)
prediction = apply_rejection(probabilities, thresholds_from_validation)
```

Table 7. Mini100 test results; test size was only 17 images [4].

Metric	SigLIP2 zero-shot	Frozen-embedding linear head
Known Top-1	100.00%	92.86%
Known Top-3	100.00%	100.00%

Metric	SigLIP2 zero-shot	Frozen-embedding linear head
Macro-F1	94.05%	88.33%
Unknown recall	100.00%	66.67%
Known acceptance	92.86%	78.57%

The zero-shot baseline outperformed the linear head and was therefore retained as the stronger experimental baseline. However, the code marks the experiment as experimental-only and records that the unknown-class metric is unstable because the test set contains only three unknown images [13]. Accordingly, the 100% Top-1 result must be interpreted only as a Mini100 experiment, not as product accuracy.

6.3 Independent dish and ingredient fine-tuning

The project subsequently explored task-specific fine-tuning. The dish classifier is single-label with an explicit unknown class, while the ingredient classifier is multi-label because multiple ingredients may be present simultaneously. Historical Phase 3A training used a two-step strategy: first train the task head while freezing SigLIP2, then unfreeze the last two vision layers for limited domain adaptation [4]. The current dish training script defaults to two head-only epochs followed by six fine-tuning epochs, batch size eight, head learning rate 2e-4, backbone learning rate 1e-5, AdamW, and label smoothing; the historical Phase 3A execution used four fine-tuning epochs, while a later mapping-aware phase used six [4], [14].

For ingredients, the project uses independent sigmoid-style outputs and an Asymmetric Loss to handle the imbalance between many absent labels and relatively few present labels. Per-label thresholds are calibrated rather than forcing one global threshold. This is appropriate for multi-label recognition but requires enough positive examples per ingredient to avoid severe false-positive or false-negative behavior.

Table 8. Historical Phase 3A semantic results [4].

Evaluation	Dish	Ingredients
Phase 3A validation	Top-1 94.55%; Top-3 98.18%; unknown recall 100%	Precision 89.87%; Recall 93.42%; F1 91.61%
Independent supported test	Top-1 66.67%; Top-3 77.78%	Precision 60.00%; core recall 30.77%
Historical decision	Dish-only staging in that snapshot	Not activated

The discrepancy between high validation scores and much weaker independent ingredient metrics is evidence of domain shift, class imbalance, and/or insufficient positive examples. A later mapping-aware ingredient experiment trained 45 labels with only two minimum positive examples in that experiment; its validation recall was high (87.68%) but precision fell to 28.54%, indicating many false positives [4].

6.4 Current semantic deployment state

Current authoritative state: CURRENT_STATE.md and MODEL_REGISTRY.md list the active semantic runtime as SigLIP2 zero-shot dish ranking plus catalog-first ingredient suggestions. Fine-tuned dish and ingredient checkpoints exist as optional assets but are not active because the dataset/quality gates were not satisfied [2], [3].

This design places the curated catalog between visual evidence and final user-facing ingredients. The runtime pipeline can still load fine-tuned models if explicitly enabled and approved, but the current model version reported by the analysis response identifies dish semantics as SigLIP2 zero-shot and ingredients as catalog evidence when those fine-tuned checkpoints are inactive [15].

7. Runtime AI Pipeline and API Orchestration

7.1 API layer and finalize-first contract

The FastAPI application is defined in src/vitamate_ai_package/api/app.py. It exposes a browser demo, a health endpoint, dishware profiles, POST /analyze, and POST /finalize. The analyze endpoint validates that the uploaded file is an image, reads it into memory, and forwards it to VitaMateAIPipeline.analyze_meal(). Finalization resolves a temporary session and validates the user-confirmed dish, ingredients, and weight [16].

Figure 3. Current finalize-first runtime pipeline.

POST /analyze → Load + validate image → Food segmentation → Capture/semantic region gate → Dish Top-3 + catalog ingredient suggestions → Optional depth/auto-weight → Create 30-min analysis session → User confirms dish + ingredients + weight → POST /finalize → Local nutrient lookup + final nutrition

7.2 Baseline image processing and segmentation

The pipeline loads the image, creates RGB/BGR representations, normalizes the selected dishware profile, invokes FoodSegmenter, obtains a boolean food mask and geometry diagnostics, and computes mask coverage. If automatic weight is skipped, the pipeline returns a stable manual-weight-required result without loading the depth model. If automatic weight is requested, MetricDepthEstimator predicts depth and DeterministicWeightEstimator executes the guarded geometric calculation [15].

7.3 Semantic region gating

Semantic prediction is not attempted blindly. The pipeline computes the number and size distribution of connected food-mask components, center offset, mask coverage, segmentation confidence, and a region-usability score. The semantic skip logic can reject missing masks, low-confidence segmentation, unsupported mask coverage, multi-container scenes, highly fragmented masks, or off-center unsupported captures [15]. This is an important reliability mechanism because semantic recognition is only meaningful when the region supplied to SigLIP2 actually represents the food.

7.4 Dish and ingredient inference

A food crop is generated from the food mask. If fine-tuned semantic models are active, their predictions can be used first. When the fine-tuned dish result is absent, unknown, or below its fallback threshold, the zero-shot classifier scores the configured dish vocabulary. Ingredient visual evidence may also be generated, but the user-facing suggestion path is catalog-first and dish-conditioned [15]. In the current deployment state,

the fine-tuned semantic checkpoints are inactive; therefore, zero-shot dish ranking and catalog-first ingredients are authoritative [2], [3].

7.5 Analyze/finalize session boundary

After /analyze, the pipeline creates an in-memory session record containing the analysis context. The session store assigns a UUID and an expiration time; the default TTL is 1,800 seconds (30 minutes). The session is explicitly ephemeral and is lost after application restart [2], [17]. This design keeps the prototype local and simple but is not a persistent production session architecture.

The analyze response includes model versions, mask summary, capture-gate reasons, dish Top-K, ingredient evidence/suggestions, weight diagnostics, and a provisional nutrition field. Final nutrition is not authoritative at this stage. /finalize accepts the confirmed dish label/ID, confirmed total weight, confirmed ingredient selection, and weight source, then returns the final nutrient breakdown after validation [1], [16].

7.6 Nutrition finalization

Nutrition is calculated deterministically from local lookup data rather than inferred by a generative model. The nutrition package contains nutrient fields for energy, macronutrients, fiber, sodium, potassium, minerals, and vitamins. The estimator can produce provisional estimates only when a valid weight is available, while the finalize-first contract requires explicit confirmation before final nutrition is returned [1], [18].

8. Detailed Guide to src/vitamate_ai_package

The src package is the reusable runtime codebase. The following table summarizes the principal packages and files confirmed in the current repository. This section is intended as a committee-oriented map: scripts explain how models are prepared and evaluated, while src explains what executes when a user sends a request.

Table 9. Practical map of the current src/vitamate_ai_package runtime package.

Runtime module/file	Committee-level responsibility
api/app.py	FastAPI application; /analyze, /finalize, health and demo routes.
pipeline/service.py	Central orchestration of segmentation, capture gates, semantics, optional weight, sessions, and responses.
pipeline/session_store.py	Thread-safe in-memory TTL store connecting analyze and finalize.
capture/validator.py	Capture-quality gates and region-usability decision support.
segmentation/yolo_segmenter.py	YOLO inference, primary/rescue handling, mask cleanup, plate-aware mask logic and fallbacks.
segmentation/common.py	Shared segmentation result structures/helpers.
classification/siglip2_classifier.py	Zero-shot SigLIP2 image/text feature scoring.
classification/finetuned_classifier.py	Optional loader/inference wrapper for fine-tuned semantic classifiers.
semantics/catalog_mapper.py	Catalog-level normalization/mapping of semantic outputs.

Runtime module/file	Committee-level responsibility
semantics/dish_classifier.py	Dish classification support logic.
semantics/dish_ranker.py	Closed-set dish ranking logic.
semantics/ingredient_resolver.py	Resolve/normalize ingredient evidence.
semantics/ingredient_suggester.py	Dish-conditioned catalog-first ingredient suggestion.
mapping/food_mapper.py	Maps raw model labels/scores into project canonical taxonomy.
depth/depth_anything_metric.py	Local Depth Anything V2 metric-depth loader and inference wrapper.
scale/resolver.py	Resolves physical cm/pixel scale from geometry/profile/manual references.
weight/estimator.py	Deterministic guarded food-weight estimator.
weight/plate_geometry.py	Dish/plate geometry representation and geometry checks.
nutrition/estimator.py	Local nutrient matching and provisional estimation support.
schemas/api.py	Pydantic request/response schemas and API contracts.
finetune/semantic_dataset.py	Reusable dataset helpers for semantic fine-tuning.
finetune/siglip_heads.py	SigLIP task heads and trainable-layer configuration.
evaluation/baseline_metrics.py	Evaluation helpers for baseline/runtime measurements.
evaluation/semantics_mini100.py	Mini100 integrity, rejection calibration and evaluation utilities.
evaluation/temporary_segmentation.py	Binary mask metrics and segmentation evaluation utilities.
config.py	Central package settings, model paths, thresholds, and runtime flags.
segmentation_registry.py	Segmentation model registry/manifest resolution.
vocab/*	Curated dish/ingredient vocabulary, density rules and nutrition/catalog assets.
utils/*	Image I/O, mask operations, device/runtime helpers and reusable utilities.

8.1 Why pipeline/service.py is the most important runtime file

VitaMateAIPipeline acts as the service coordinator. Its constructor loads density rules and vocabularies, constructs segmentation, depth, weight, nutrition, mapping, and session components, and defers semantic-model loading until needed. The analyze_meal method executes the baseline, computes semantic region

metrics, conditionally predicts semantics, optionally re-estimates weight using semantic density evidence, applies capture/decision logic, stores a session, and returns a structured AnalysisResponse [15].

8.2 Why config.py and manifests matter

A model path is not considered active merely because a checkpoint exists. Package configuration and active manifests determine which model is loaded, while MODEL_REGISTRY.md records the authoritative deployment interpretation. This is a strong reproducibility practice because it separates artifact existence from deployment approval [3].

9. Tools, Libraries, and Execution Environment

The AI package is implemented in Python and uses a local execution model. The project dependency specification reviewed earlier in the repository includes FastAPI, Uvicorn, python-multipart, Pydantic, NumPy, OpenCV, Pandas, Pillow, PyTorch, torchvision, Ultralytics, Transformers 5.5.4, huggingface_hub, safetensors, and Matplotlib. The current source code additionally uses scikit-learn in evaluation scripts for confusion matrices and classification metrics. Core model assets are stored locally, which allows the service to run without sending meal images to a remote inference API.

Table 10. Principal software tools and their roles.

Tool / library	Role in VitaMate
Python	Primary implementation language and experiment orchestration.
PyTorch / torchvision	Neural-network training/inference, GPU/CPU device execution.
Ultralytics YOLO	Food segmentation training and runtime mask prediction [19].
Hugging Face Transformers	SigLIP2 processor/model loading and vision-language inference [20].
Depth Anything V2	Optional monocular metric-depth estimation for auto-weight [9].
OpenCV	Mask resizing, morphology, connected components, image processing.
NumPy	Mask arithmetic, depth/volume calculations, metrics.
Pillow	Image loading/cropping and semantic image handling.
Pandas / CSV utilities	Dataset manifests and offline experiment analysis.
scikit-learn	Confusion matrices, F1/recall and evaluation support.
FastAPI	HTTP API implementation.
Pydantic	Typed request/response schemas and validation.

Tool / library	Role in VitaMate
Uvicorn	Local ASGI server for the FastAPI service.
Matplotlib	Evaluation plots such as confusion matrices.
safetensors / huggingface_hub	Local model weight and model-asset support.
pytest	Repository verification and automated tests.

9.1 External model and dataset foundations

Three external foundations are especially relevant. Nutrition5k provides a research precedent for combining images, measured component weights, depth and nutrition annotations [7]. UECFoodPix/UECFoodPixComplete provide large-scale food segmentation masks and were used historically in segmentation adaptation experiments [21]. SigLIP2 provides the vision-language semantic backbone [8], while Depth Anything V2 provides the optional monocular metric-depth component [9].

10. Experimental Results and Current Runtime State

10.1 Historical experimental evidence

Table 11. Selected historical experiment results from the project training record [4].

Stage	Key result	Interpretation
Phase 0 governance	60 valid images; 59 valid human masks; 40/10/9 human train/val/test	Data readiness achieved for controlled project experiments.
Human-mask segmentation	seed17: mean IoU 90.74%, Dice 95.01%, recall 94.96% on 9 images	Strong small-test candidate; historical result only.
Weight calibration	8/9 valid after guard; MAE 163.21 g; MAPE 36.40%	Automatic weight remained too inaccurate for trusted default.
Mini100 SigLIP2 zero-shot	Known Top-1/Top-3 100% on 17 test images	Useful proof-of-concept, not product accuracy.
Phase 3A dish test	Top-3 77.78% on supported independent test	Met historical dish staging target in that snapshot.
Phase 3A ingredients	Precision 60%; core recall 30.77%	Failed ingredient promotion targets.
Historical 35-image release eval	35/35 evaluated; 0 crashes; manual-finalize readiness 91.43%	Stable pipeline snapshot but semantic accuracy below product gates.

10.2 Current authoritative runtime state

The current GitHub CURRENT_STATE.md should be used for deployment claims. It reports the following latest saved runtime measurements and explicitly states that they demonstrate implementation stability but do not yet satisfy semantic reliability gates [2].

Table 12. Current authoritative runtime metrics from CURRENT_STATE.md [2].

Current saved runtime metric	Value
Segmentation runtime completion	35 / 35; 0 crashes
Mean segmentation confidence	53.72%
Mean region usability	76.52%
Full pipeline semantic-ready or better	22 / 35
Partial	13 / 35
Protocol-clean manual-finalize readiness	80%
Protocol-clean canonical Top-3	100%
Overall closed-set Top-1	28.57%
Overall closed-set Top-3	42.86%
Ingredient precision	35.98%
Ingredient core recall	14.57%
Finalize mapping success	62.86%

These values differ from the older 35-image release-evaluation snapshot in the training-history document (for example, 32 semantic-region-ready and 91.43% manual-finalize readiness). Because the two sources represent different project states, they are intentionally not averaged or merged. The current-state document is used for present deployment claims; the older table is retained as a historical experiment record.

11. Reliability, Promotion Governance, and Reproducibility

11.1 Human-in-the-loop reliability design

- No final nutrition is returned solely from /analyze; user confirmation is required before /finalize.
- Manual weight remains available when automatic weight is skipped or rejected by guards.
- Semantic predictions are gated by segmentation confidence, mask coverage, component structure, and capture geometry.
- Catalog-first ingredients constrain visually implausible or out-of-taxonomy ingredient suggestions.
- Unknown/uncertain semantic cases can be rejected rather than forced into a known class.
- Primary/rescue segmentation and promotion manifests provide rollback and controlled activation.

11.2 Reproduction commands

The project training-history document records a reproducible stage order. Exact CLI arguments should still be checked with each script's --help because local paths and manifests may vary [4].

Representative reproducibility sequence (paths/arguments omitted for clarity).

```
# Phase 0
python scripts/prepare_vitamate_phase0_governance.py

# Phase 1 segmentation
python scripts/annotate_vitamate_human_masks.py
python scripts/prepare_vitamate_phase1_segmentation_dataset.py
python scripts/train_vitamate_final_segmentation.py --preflight-only
python scripts/train_vitamate_final_segmentation.py
python scripts/evaluate_segmentation_on_vitamate_human_masks.py

# Phase 2 weight
python scripts/prepare_vitamate_phase2_weight_dataset.py
python scripts/review_vitamate_phase2_weight_protocol.py
python scripts/calibrate_vitamate_weight_from_baseline.py

# Semantics / release
python scripts/prepare_vitamate_final_semantic_dataset.py
python scripts/train_vitamate_dish_classifier.py
python scripts/train_vitamate_ingredient_classifier.py
python scripts/evaluate_vitamate_semantic_models.py
python scripts/run_vitamate_eval_suite.py
python scripts/evaluate_vitamate_release_gate.py
```

11.3 Runtime verification

The repository README recommends installing the AI package dependencies, running scripts/check_ai_package_ready.py, starting run_local_api.bat, then verifying with pytest and scripts/smoke_test_ai_package.py [1]. Model and dataset files are kept local and are not committed to Git, so reproducibility also depends on preserving the exact manifests, hashes and checkpoints described by the registry.

12. Limitations and Threats to Validity

12.1 Small independent evaluation sets

The strongest historical segmentation result is based on only nine human-mask test images. The Mini100 semantic test contains only 17 samples, including only three unknown images. These sizes are insufficient for broad claims about generalization, per-class reliability, or open-world robustness. Confidence intervals would also be wide for many metrics.

12.2 Domain shift and class imbalance

Semantic validation scores were substantially higher than independent or broader release-evaluation scores. This pattern indicates that the evaluation distribution differs from training/validation or that the class coverage is insufficient. Ingredient recognition is particularly vulnerable because many labels have few positive examples, while every image contributes many negative labels.

12.3 Monocular weight uncertainty

Single-image weight estimation compounds errors from segmentation, plate geometry, physical scale, monocular depth, density selection and shape approximation. The historical 163.21 g MAE supports the

decision not to present auto-weight as reliable. The current 35-image suite cannot independently validate automatic weight because measured weights are unavailable for those images [2], [4].

12.4 Deployment scope

The service is a controlled MVP, not an open-world food-recognition system. It assumes a supported capture protocol, a curated dish catalog, explicit confirmation, local sessions, and local assets. The current state explicitly reports that fine-tuned semantic models are inactive and that the semantic reliability gate is not yet satisfied [2].

12.5 Source-state differences

The internal training history and current repository state are both valid sources for different purposes. Training-history tables document what happened in earlier experiments; MODEL_REGISTRY.md and CURRENT_STATE.md document what the service currently considers active. This report therefore avoids presenting a historical promotion decision as the current deployed state.

13. Conclusion

The VitaMate AI Food Analysis Service is best characterized as a modular, locally executed, human-in-the-loop computer vision pipeline rather than a single end-to-end neural model. Its implementation separates food segmentation, semantic recognition, geometric weight estimation, curated mapping, and nutrition finalization into independently testable modules. The training history shows systematic experimentation with data governance, controlled YOLO fine-tuning, independent human-mask evaluation, weight calibration using measured ground truth, zero-shot SigLIP2, partial semantic fine-tuning, mapping-aware ingredient constraints, and release gates. The current runtime intentionally adopts the more conservative configuration: active official/rescue segmentation, SigLIP2 zero-shot closed-set semantics, catalog-first ingredient suggestions, manual weight as the reliable default, optional Depth Anything V2 only for diagnostic automatic weight, and explicit user confirmation before final nutrition [1]-[3].

From an academic software-engineering perspective, the project's strongest qualities are its separation of experimental and deployment states, explicit model manifests, independently reported failure modes, deterministic finalization logic, and refusal to promote weak ingredient or automatic-weight behavior as a reliable product feature. Its principal limitations are the small independent test sets, class imbalance, domain shift, and geometric uncertainty of monocular weight estimation. Further progress should prioritize additional human-corrected masks, balanced dish/ingredient collection, measured-weight test images under the supported capture protocol, and strict independent promotion gates before activating new checkpoints.

Appendix A. Important Training Logic (Simplified)

A.1 Segmentation two-stage fine-tuning

Conceptual reconstruction of the current final-segmentation training script [5].

```
initializer = official_foodv1_checkpoint
for seed in [17, 29]:
    warm = YOLO(initializer).train(freeze=15, epochs=5, lr0=1e-4, optimizer='AdamW')
    final = YOLO(warm.best).train(freeze=0, epochs=15, lr0=2e-5, optimizer='AdamW')
    record hashes + dataset fingerprint + checkpoints
```

A.2 Segmentation evaluation

Conceptual reconstruction of human-mask evaluation [6].

```
prediction_mask = union(resized_YOLO_masks > 0.5)
metrics = compare(prediction_mask, human_mask)
summary = mean_and_median(IoU, Dice, Precision, Recall, leakage, latency)
rank candidates by mean IoU
```

A.3 Weight calibration

Conceptual reconstruction of shape-factor calibration [10].

```
x = raw_volume_cm3 * density
y = measured_weight_g
least_squares_factor = sum(x*y) / sum(x*x)
median_ratio_factor = median(y/x)
select only if validation MAE improves by required margin
```

A.4 SigLIP2 zero-shot experiment

Conceptual reconstruction of Mini100 zero-shot evaluation [13].

```
text_features = normalize(SigLIP2.encode_text(prompts))
image_features = normalize(SigLIP2.encode_image(image))
scores = similarity(image_features, text_features)
thresholds = calibrate_unknown_rejection(validation)
metrics = evaluate_once(test)
```

A.5 Partial semantic fine-tuning

Simplified semantic fine-tuning strategy documented in the project history [4].

```
Stage 1: freeze SigLIP2 -> train new task head
Stage 2: unfreeze final vision layers -> train head + selected backbone layers at lower LR
Dish: single-label + unknown
Ingredients: multi-label + asymmetric loss + per-label thresholds
```

Appendix B. Important Runtime Logic (Simplified)

B.1 Analyze request

Runtime orchestration based on api/app.py and pipeline/service.py [15], [16].

```
/analyze(image)
  -> run_baseline()
  -> food segmentation
  -> semantic region gate
  -> dish Top-K + catalog suggestions
  -> optional depth/weight
  -> create temporary session
  -> return AnalysisResponse (not final nutrition)
```

B.2 Automatic weight

Runtime automatic-weight logic based on depth and weight modules [11], [12].

```
if auto_weight_mode == 'skip':
    request manual weight
else:
    depth = DepthAnythingV2(image)
    scale = resolve_physical_scale(...)
    food_height = max(plate_depth - pixel_depth, 0)
    raw_volume = sum(food_height * pixel_area)
    weight = raw_volume * shape_factor * density
    reject if guards fail
```

B.3 Finalize request

Finalize-first contract [16], [17], [18].

```
/finalize(session_id, confirmed_dish, confirmed_ingredients, confirmed_weight)
  -> load 30-minute session
  -> validate confirmed selections
  -> map to local nutrient database
  -> compute finalized nutrient breakdown
  -> return FinalizeResponse
```

References

[1] VitaMate AI Food Analysis Service, README.md, public GitHub repository, reviewed 18 Aug. 2026. https://github.com/amenahzitoun-ENG/AI-Food-Analysis

[2] VitaMate AI Food Analysis Service, CURRENT_STATE.md, public GitHub repository, reviewed 18 Aug. 2026. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/CURRENT_STATE.md

[3] VitaMate AI Food Analysis Service, MODEL_REGISTRY.md, public GitHub repository, reviewed 18 Aug. 2026. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/MODEL_REGISTRY.md

[4] VitaMate AI Food Analysis Project, COMPLETE_TRAINING_HISTORY_AR, internal project technical record, Aug. 2026.

[5] VitaMate project source, scripts/train_vitamate_final_segmentation.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/scripts/train_vitamate_final_segmentation.py

[6] VitaMate project source, scripts/evaluate_segmentation_on_vitamate_human_masks.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/scripts/evaluate_segmentation_on_vitamate_human_masks.py

[7] Q. Thames et al., "Nutrition5k: Towards Automatic Nutritional Understanding of Generic Food," 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021. https://arxiv.org/abs/2103.03375

[8] M. Tschannen et al., "SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features," arXiv:2502.14786, 2025. https://arxiv.org/abs/2502.14786

[9] L. Yang et al., "Depth Anything V2," arXiv:2406.09414, 2024. https://arxiv.org/abs/2406.09414

[10] VitaMate project source, scripts/calibrate_vitamate_weight_from_baseline.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/scripts/calibrate_vitamate_weight_from_baseline.py

[11] VitaMate project source, src/vitamate_ai_package/depth/depth_anything_metric.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/src/vitamate_ai_package/depth/depth_anything_metric.py

[12] VitaMate project source, src/vitamate_ai_package/weight/estimator.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/src/vitamate_ai_package/weight/estimator.py

[13] VitaMate project source, scripts/evaluate_siglip2_mini100_zero_shot.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/scripts/evaluate_siglip2_mini100_zero_shot.py

[14] VitaMate project source, scripts/train_vitamate_dish_classifier.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/scripts/train_vitamate_dish_classifier.py

[15] VitaMate project source, src/vitamate_ai_package/pipeline/service.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/src/vitamate_ai_package/pipeline/service.py

[16] VitaMate project source, src/vitamate_ai_package/api/app.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/src/vitamate_ai_package/api/app.py

[17] VitaMate project source, src/vitamate_ai_package/pipeline/session_store.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/src/vitamate_ai_package/pipeline/session_store.py

[18] VitaMate project source, src/vitamate_ai_package/nutrition/estimator.py, public GitHub repository. https://github.com/amenahzitoun-ENG/AI-Food-Analysis/blob/main/src/vitamate_ai_package/nutrition/estimator.py

[19] Ultralytics, “Instance Segmentation with Ultralytics YOLO,” official documentation. https://docs.ultralytics.com/tasks/segment/

[20] Hugging Face, “SigLIP2,” Transformers documentation. https://huggingface.co/docs/transformers/en/model_doc/siglip2

[21] K. Okamoto and K. Yanai, “UEC-FoodPix Complete: A Large-Scale Food Image Segmentation Dataset,” ICPR Workshops, 2020/2021. https://mm.cs.uec.ac.jp/uecfoodpix/